Revista Nacional de las Ciencias para Estudiantes
Vol. 2, No. 1 (2025). Pp. 28–39 | **ISSN: 3087-2839**

Interacciones Gravitacionales en Sistemas Galácticos

Carbonell, Amir[1]

INFORMACIÓN DEL ARTÍCULO

Historial:
Recibido: 10 de Nov. 2025
Aceptado: 08 de May. 2025
Publicado: 8 de mayo de 2026

Área temática:
Astrofísica Computacional

ISSN: 3087-2839

Sitio web:
www.revistacienciasestudiantes.com

Tipo de artículo:
Artículo Original

Licencia Creative Commons
Atribución 4.0 Internacional

Resumen

Este trabajo presenta un código en Python diseñado para simular interacciones gravitacionales en sistemas galácticos. El código utiliza métodos numéricos, específicamente la integración de ecuaciones diferenciales ordinarias (ODE), para modelar el movimiento de galaxias y estrellas bajo la influencia de la gravedad. Se describen detalladamente los parámetros físicos y de animación utilizados en las simulaciones.
Palabras clave: Simulaciones, galaxias, soluciones numéricas de ODEs

Abstract

This work presents a Python code designed to simulate gravitational interactions in galactic systems. The code uses numerical methods, specifically the integration of ordinary differential equations (ODE), to model the motion of galaxies and stars under the influence of gravity. The physical and animation parameters used in the simulations are described in detail.
Keywords: Simulations, galaxies, numerical solutions of ODEs

1. Alumno, 3ro medio Liceo José Victorino Lastarria

1 Introducción

En este estudio, se presenta un código en Python con el propósito principal de simular las complejas interacciones gravitacionales presentes en sistemas galácticos. El enfoque de este código recae en la aplicación de métodos numéricos, particularmente la integración de ecuaciones diferenciales ordinarias (ODE), para modelar de manera precisa el movimiento de galaxias y estrellas bajo la influencia gravitacional. A diferencia de las aproximaciones analíticas, las simulaciones numéricas permiten una representación más realista y detallada de los fenómenos astrofísicos, especialmente en escenarios donde las fuerzas de marea desempeñan un papel crucial.

El propósito principal de esta simulación es extrapolar nuestro conocimiento sobre las fuerzas de marea a través de la aplicación específica de estas fuerzas en el contexto de la colisión de galaxias. Este enfoque busca proporcionar una comprensión más profunda de cómo las interacciones gravitacionales afectan la dinámica de sistemas galácticos durante colisiones, donde las fuerzas de marea pueden desencadenar fenómenos como la formación de estructuras a gran escala, la deformación de galaxias y la redistribución de masa.

En el transcurso de este documento, se detallarán los parámetros físicos y de animación utilizados en las simulaciones, permitiendo así una comprensión completa de las condiciones iniciales y los factores que influyen en la evolución de los sistemas galácticos simulados. La capacidad de ajustar estos parámetros brinda flexibilidad al investigador para explorar diversos escenarios y condiciones iniciales, contribuyendo a la robustez y versatilidad del código.

En resumen, este código no solo se presenta como una herramienta de simulación, sino como un medio para avanzar en nuestra comprensión de las fuerzas de marea en la colisión de galaxias, utilizando métodos numéricos para capturar con precisión las complejidades de estos eventos astrofísicos.

2 Descripción del Código

El código en Python proporciona una simulación detallada de interacciones gravitacionales en sistemas galácticos, con un enfoque específico en la colisión de galaxias. A continuación, se detallan los componentes y funcionalidades clave del código:

1. Parámetros Físicos:
   - Constantes Gravitacionales: El código utiliza la constante de Gravitación Universal (G) para modelar las fuerzas gravitacionales entre las galaxias y estrellas.
   - Masa de Galaxias: Se definen masas para galaxias solitarias (M) y para dos galaxias en interacción (M1 y M2).
   - Dirección de Rotación: La variable clockwise determina si las galaxias rotan en sentido horario o antihorario durante la colisión.
2. Parámetros de Animación y Ploteo:
   - Rango de Ploteo (`xy_max`): Define el rango del gráfico para visualizar las posiciones de las galaxias y estrellas.
   - Número de Pasos de Tiempo (`N`): Controla la resolución temporal de la simulación.
   - Duración de la Animación (`animation_length`): Indica la duración total del video generado.
   - Directorios de Almacenamiento: Establece directorios para guardar figuras y videos generados durante la simulación.
3. Funciones de Ecuaciones Diferenciales Ordinarias (ODE):

- `ODE_single_galaxy`: Define las ODE para el movimiento de una galaxia solitaria.
- `ODE_galaxy_motion`: Define las ODE para el movimiento de dos galaxias en interacción.
- `ODE_double_galaxy`: Define las ODE para el movimiento de galaxias y estrellas en un sistema doble.

4. Funciones de Integración Temporal:
   - `integrate_single_galaxy`: Utiliza odeint para integrar las ODE de una galaxia solitaria y devuelve las trayectorias.
   - `integrate_galaxy_motion`: Realiza la integración temporal para el movimiento de dos galaxias en interacción.
   - `integrate_double_galaxy`: Integra las ecuaciones para el movimiento de galaxias y estrellas en un sistema doble.
5. Funciones para Generar Condiciones Iniciales (`SIV_`):
   - `SIV_single_galaxy_ring`: Genera condiciones iniciales para una galaxia solitaria en forma de anillo.
   - `SIV_single_galaxy_random`: Genera condiciones iniciales para una galaxia solitaria con posiciones y velocidades aleatorias.
   - `SIV_double_galaxy_ring`: Genera condiciones iniciales para un sistema doble de galaxias en forma de anillo.
   - `SIV_galaxy_motion`: Genera condiciones iniciales para el movimiento de dos galaxias en interacción.
6. Funciones de Obtención de Datos (`GetData_`):
   - `GetData_single_galaxy`: Obtiene datos de trayectorias para una galaxia solitaria.
   - `GetData_galaxy_motion`: Obtiene datos de trayectorias para dos galaxias en interacción.
   - `GetData_double_galaxy`: Obtiene datos de trayectorias para galaxias y estrellas en un sistema doble.
7. Funciones de Animación y Ploteo:
   - `update_func`: Actualiza los datos para cada fotograma de la animación.
   - `animate`: Crea y muestra la animación de la evolución temporal del sistema.
   - `plot_figures`: Genera y guarda figuras estáticas en momentos específicos de la simulación.
8. Funciones de Utilidad:
   - `theta_for_closest_approach`: Calcula el ángulo inicial para el acercamiento más cercano entre dos galaxias.
   - `datetime`: Utilizado para etiquetar archivos de salida con marcas de tiempo.

El código se estructura de manera modular, facilitando la comprensión y la adaptación a diferentes configuraciones de simulación. La utilización de funciones y parámetros configurables proporciona flexibilidad para explorar una variedad de escenarios astrofísicos y condiciones iniciales.

3 Tipos de Simulación

El código presenta cuatro tipos principales de simulación, cada uno diseñado para modelar diferentes escenarios de interacciones gravitacionales en sistemas galácticos:

1. Simulación de Galaxia Solitaria en Anillo:
   - La función `animation_single_galaxy_ring` utiliza condiciones iniciales generadas en forma de anillo para simular el movimiento de una galaxia solitaria.
   - Permite explorar cómo una galaxia única evoluciona cuando las estrellas y los planetas están distribuidos en un patrón circular.
2. Simulación de Galaxia Solitaria con Posiciones y Velocidades Aleatorias:
   - La función `animation_single_galaxy_random` utiliza condiciones iniciales generadas aleatoriamente para simular el movimiento de una galaxia solitaria.
   - Ofrece la posibilidad de estudiar el comportamiento de una galaxia con distribuciones iniciales no estructuradas.
3. Simulación de Colisión entre Dos Galaxias:
   - La función `animation_galaxy_motion` simula la interacción gravitacional entre dos galaxias, modelando su movimiento y colisión.
   - Permite explorar cómo la fuerza gravitacional afecta el comportamiento de dos sistemas galácticos que se acercan.
4. Simulación de Colisión entre Dos Galaxias con Estrellas en un Anillo:
   - La función `animation_double_galaxy` combina la simulación de la colisión entre dos galaxias con la presencia de estrellas distribuidas en un anillo en ambas galaxias.
   - Permite estudiar la dinámica compleja de un sistema galáctico doble con la influencia adicional de estrellas circundantes.

Cada tipo de simulación ofrece una perspectiva única sobre las fuerzas gravitacionales y sus efectos en la evolución de los sistemas galácticos. Los parámetros iniciales, como las masas, las posiciones y las velocidades, pueden ajustarse para explorar una variedad de condiciones iniciales y escenarios astrofísicos. Además, la capacidad de visualización a lo largo del tiempo y la generación de figuras estáticas en momentos específicos facilitan el análisis detallado de la evolución de estos sistemas bajo la influencia de las fuerzas gravitacionales.

4 Teoría Matemática detrás del Código

1. Ecuaciones Diferenciales Ordinarias (ODE):
   - La mecánica celeste utiliza ODE para describir el movimiento de partículas bajo la influencia de fuerzas gravitacionales.
   - Para un objeto en el espacio, la posición y la velocidad se modelan como funciones del tiempo a través de ecuaciones diferenciales.
   - La simulación utiliza la función odeint de scipy para integrar numéricamente estas ecuaciones y obtener soluciones aproximadas.
2. Ley de Gravitación Universal de Newton:

- La fuerza gravitacional entre dos objetos se rige por la Ley de Gravitación Universal:

$$F = G\frac{m_1 m_2}{r^2}$$

  donde F es la fuerza, G es la constante gravitacional, m_1 y m_2 son las masas de los objetos, y r es la distancia entre ellos.

- Las componentes de esta fuerza se incorporan en las ecuaciones de movimiento para cada objeto en el sistema.

3. Ecuaciones de Movimiento para Galaxias y Estrellas:
   - La posición y la velocidad de cada objeto se modelan mediante sistemas de ecuaciones diferenciales acopladas.
   - Para una galaxia solitaria, las ecuaciones consideran la interacción gravitacional entre las estrellas dentro de la galaxia.
   - En el caso de la colisión de dos galaxias, se suman las fuerzas gravitacionales de ambas.
4. Condiciones Iniciales y Soluciones Numéricas:
   - Las condiciones iniciales, como las posiciones y velocidades iniciales de los objetos celestes, se establecen para resolver las ODE.
   - El método de integración numérica permite obtener soluciones aproximadas para la evolución temporal del sistema.
5. Análisis de Colisiones y Distancias de Aproximación:
   - Para simular colisiones galácticas, se calculan las distancias de aproximación mínima entre las galaxias.
   - Se utiliza la teoría de perturbaciones para evaluar el impacto de estas distancias en la dinámica del sistema.
6. Parámetros Físicos y Dinámica del Sistema:
   - La masa total, la distribución espacial de masas y las velocidades iniciales afectan la dinámica del sistema.
   - La elección de parámetros influye en la estabilidad, órbitas y resultados de la simulación.
7. Generación de Trayectorias y Visualización:
   - Las trayectorias de los objetos se generan mediante la solución numérica de las ecuaciones de movimiento.
   - La visualización de las trayectorias a lo largo del tiempo proporciona información sobre la evolución del sistema.

La simulación se apoya en principios matemáticos sólidos, empleando herramientas numéricas para representar el comportamiento dinámico de sistemas galácticos bajo la influencia gravitacional.

5 Código

```
# -- coding: utf-8 --
import numpy as np
import numpy.random as rand
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from numpy import cos, sin, pi, sqrt
from scipy.integrate import odeint
```

```
from scipy.special import erf
import time
import datetime
import os
# Parametros Fisicos
G = 0.2 # Constante de Gravitacion Universal
M = 4 # Masa de una Galaxia solitaria
M1 = 3.5 # Masa1 de sistema de Galaxias
M2 = 2 # Masa2 de sistema de Galaxias
# = REQUISITO =#
clockwise = True # True -> las galaxias rotan en sentido horario, False -> lo contrario
# Parametros de Animacion y Ploteo
xy_max = 50 # rango del ploting
N = 500 # Numero de pasos de tiempo para integrar
animation_length = 10 # Duracion del video en segundos, escala N para que la animacion sea asi de larga
t_max = 500
t_selection = [0, 50, 100, 150, 200, 300, 400, 499] # Tiempos en las que desea trazar (debe ser un numero
    entero y < t_max)
current_directory = os.getcwd()
figure_directory = os.path.join(current_directory, 'Figures/')
movie_directory = os.path.join(current_directory, 'Movies/')
if not os.path.exists(figure_directory):
    os.mkdir(figure_directory)
if not os.path.exists(movie_directory):
    os.mkdir(movie_directory)
plotfigures = True
savemovie = True
def ODE_single_galaxy(r, t):
    return [r[2], r[3], -G * M * r[0] / (r[0]**2 + r[1]**2)**(3/2), -G * M * r[1] / (r[0]**2 + r[1]**2)
        **(3/2)]
def ODE_galaxy_motion(R, t):
    x, y = R[0] - R[4], R[1] - R[5]
    return [R[2], R[3], -G * M2 * x / (x**2 + y**2)**(3/2),
            -G * M2 * y / (x**2 + y**2)**(3/2), R[6], R[7],
            G * M1 * x / (x**2 + y**2)**(3/2),
            G * M1 * y / (x**2 + y**2)**(3/2)]
def ODE_double_galaxy(r, t, R, t_array):
    index = np.searchsorted(t_array, t)
    cgp = [] # Posicion actual de la galaxia
    for i in range(4):
        cgp.append(R[i][index - 1])
    x1, y1, x2, y2 = r[0] - cgp[0], r[1] - cgp[1], r[0] - cgp[2], r[1] - cgp[3]
    return [r[2], r[3],
            -G * M1 * x1 / (x1**2 + y1**2)**(3/2) + -G * M2 * x2 / (x2**2 + y2**2)**(3/2),
            -G * M1 * y1 / (x1**2 + y1**2)**(3/2) + -G * M2 * y2 / (x2**2 + y2**2)**(3/2)]
def integrate_single_galaxy(t, r0):
    r = odeint(ODE_single_galaxy, r0, t)
    return r[:, 0], r[:, 1]
def integrate_galaxy_motion(t, R0):
    r = odeint(ODE_galaxy_motion, R0, t)
    return r[:, 0], r[:, 1], r[:, 4], r[:, 5]
def integrate_double_galaxy(t, r0, R, t_array):
    r = odeint(ODE_double_galaxy, r0, t, args=(R, t_array))
    return r[:, 0], r[:, 1]
def SIV_single_galaxy_ring(r, n):
```

```
    r0 = []
    for j in range(len(n)):
        for i in range(n[j]):
            r0.append([r[j] * cos(i * 2 * pi / n[j]), r[j] * sin(i * 2 * pi / n[j]),
                       sqrt(G * M / r[j]) * sin(i * 2 * pi / n[j]),
                       -sqrt(G * M / r[j]) * cos(i * 2 * pi / n[j])])
    return r0
def SIV_single_galaxy_random(n, max_r):
    r0 = []
    for i in range(n):
        R = max_r * rand.random()
        theta = 2 * pi * rand.random()
        v = sqrt(G * M / R) + 0.1 * rand.uniform(-1, 1) * sqrt(G * M / R)
        delta = rand.uniform(-pi / 6, pi / 6)
        r0.append([R * cos(theta), R * sin(theta), v * sin(theta + delta), -v * cos(theta + delta)])
    return r0
def SIV_double_galaxy_ring(r, n, R0):
    k = (1 if clockwise == True else -1)
    r0 = []
    for j in range(len(n)):
        for i in range(n[j]):
            r0.append([r[j] * cos(i * 2 * pi / n[j]) + R0[0], r[j] * sin(i * 2 * pi / n[j]) + R0[1],
                       k * sqrt(G * M1 / r[j]) * sin(i * 2 * pi / n[j]) + R0[2],
                       -k * sqrt(G * M1 / r[j]) * cos(i * 2 * pi / n[j]) + R0[3]])
    for j in range(len(n)):
        for i in range(n[j]):
            r0.append([r[j] * cos(i * 2 * pi / n[j]) + R0[4], r[j] * sin(i * 2 * pi / n[j]) + R0[5],
                       k * sqrt(G * M2 / r[j]) * sin(i * 2 * pi / n[j]) + R0[6],
                       -k * sqrt(G * M2 / r[j]) * cos(i * 2 * pi / n[j]) + R0[7]])
    return r0
def SIV_galaxy_motion(s_init, s_closest):
    h = s_init / (2 * sqrt(2))
    theta = theta_for_closest_approach(s_init, s_closest)
    print("Starting angle (deg): %.1f" % (theta * 180 / pi))
    EffMass = 2 * M1 * M2 / (M1 + M2)
    v = sqrt(G * EffMass / (h * 2 * sqrt(2)))
    return [-h, h, v * cos(theta), -v * sin(theta), h, -h, -v * cos(theta), v * sin(theta)]
def GetData_single_galaxy(r0):
    t = np.linspace(0, t_max, N)
    x, y = integrate_single_galaxy(t, r0[0])
    for i in range(len(r0) - 1):
        datax, datay = integrate_single_galaxy(t, r0[i + 1])
        x = np.column_stack((x, datax))
        y = np.column_stack((y, datay))
        print("%.1f %%" % (100 * i / len(r0)), end="\r")
    return x, y
def GetData_galaxy_motion(r0):
    t = np.linspace(0, t_max, N)
    x1, y1, x2, y2 = integrate_galaxy_motion(t, r0)
    return x1, y1, x2, y2
def GetData_double_galaxy(r0, R_galaxies):
    t = np.linspace(0, t_max, N)
    x, y = integrate_double_galaxy(t, r0[0], R_galaxies, t)
    for i in range(len(r0) - 1):
        datax, datay = integrate_double_galaxy(t, r0[i + 1], R_galaxies, t)
```

```
        x = np.column_stack((x, datax))
        y = np.column_stack((y, datay))
        print("%.1f %%" % (100 * i / len(r0)), end="\r")
    return x, y
def animation_single_galaxy_ring(Radii, Numbers):
    R0 = SIV_single_galaxy_ring(Radii, Numbers)
    datax, datay = GetData_single_galaxy(R0)
    animate(datax, datay)
    plot_figures(datax, datay)
    return
def animation_single_galaxy_random(r_max, n_total):
    R0 = SIV_single_galaxy_random(n_total, r_max)
    datax, datay = GetData_single_galaxy(R0)
    animate(datax, datay)
    plot_figures(datax, datay)
    return
def animation_galaxy_motion(s_init, s_closest):
    R0 = SIV_galaxy_motion(s_init, s_closest)
    R_galaxies = GetData_galaxy_motion(R0)
    datax = np.column_stack((R_galaxies[0], R_galaxies[2]))
    datay = np.column_stack((R_galaxies[1], R_galaxies[3]))
    animate(datax, datay)
    plot_figures(datax, datay)
    return
def animation_double_galaxy(Radii, Numbers, s_init, s_closest):
    R0_galaxies = SIV_galaxy_motion(s_init, s_closest)
    R_galaxies = GetData_galaxy_motion(R0_galaxies)
    R0_stars = SIV_double_galaxy_ring(Radii, Numbers, R0_galaxies)
    datax, datay = GetData_double_galaxy(R0_stars, R_galaxies)
    animate(datax, datay)
    plot_figures(datax, datay)
    return
# OTHER FUNCTIONS
def theta_for_closest_approach(s_init, s_closest):
    Theta = [pi * i / 180 for i in range(46)]
    h = s_init / (2 * sqrt(2))
    S = []
    for theta in Theta:
        EffMass = 2 * M1 * M2 / (M1 + M2)
        initial_galaxy_data = [-h, h, sqrt(G * EffMass / (h * 2 * sqrt(2))) * cos(theta),
                               -sqrt(G * EffMass / (h * 2 * sqrt(2))) * sin(theta),
                               h, -h, -sqrt(G * EffMass / (h * 2 * sqrt(2))) * cos(theta),
                               sqrt(G * EffMass / (h * 2 * sqrt(2))) * sin(theta)]
        R_gal = GetData_galaxy_motion(initial_galaxy_data)
        galaxy_separation = sqrt((R_gal[0] - R_gal[2])**2 + (R_gal[1] - R_gal[3])**2)
        S.append(abs(s_closest - np.min(galaxy_separation)))
    i = S.index(min(S))
    return pi * i / 180
def update_func(num, datax, datay, line):
    line.set_data(datax[num, :], datay[num, :])
    return line,
def animate(datax, datay):
    fig1 = plt.figure()
    line, = plt.plot([], [], ".", markersize=1.5)
    plt.xlim(-xy_max, xy_max)
```

```
    plt.ylim(-xy_max, xy_max)
    plt.xlabel('x')
    plt.ylabel('y')
    plt.gca().set_aspect('equal')
    anim = animation.FuncAnimation(fig1, update_func, datax.shape[0], fargs=(datax, datay, line),
                                   interval=1000 * animation_length / N, blit=True)
    print("Computing time = %.1f s" % (time.time() - T0))
    if savemovie:
        anim.save(movie_directory + '%s.mp4' % datetime.datetime.now().strftime("%I%M%p%B%d"))
    plt.show()
def plot_figures(datax, datay):
    if plotfigures:
        fig, axs = plt.subplots(2, 4, figsize=(20, 10))
        axs = axs.flatten()
        for idx, t in enumerate(t_selection):
            num = int(round((t / t_max) * (N - 1)))
            axs[idx].plot(datax[num, :], datay[num, :], ".", markersize=1.5)
            axs[idx].set_xlim(-xy_max, xy_max)
            axs[idx].set_ylim(-xy_max, xy_max)
            axs[idx].set_title("Time = %.1f" % t)
            axs[idx].set_aspect('equal')
            if idx >= 4:
                axs[idx].set_xlabel("x")
            if idx % 4 == 0:
                axs[idx].set_ylabel("y")
        plt.tight_layout()
        plt.savefig(figure_directory + '%s_composite.png' % datetime.datetime.now().strftime("%I%M%p%B%d"),
                    bbox_inches='tight', dpi=500)
T0 = time.time()
# animation_single_galaxy_ring([2, 3, 4, 5, 6], [24, 36, 48, 60, 72])
# animation_single_galaxy_random(6, 240)
# animation_galaxy_motion(40, 10)
animation_double_galaxy([2, 3, 4, 5, 6], [24, 36, 48, 60, 72], 40, 10)
```

6 Imágenes Realizadas con el Simulador

Algunos ejemplos que fueron renderizados con el simulador.

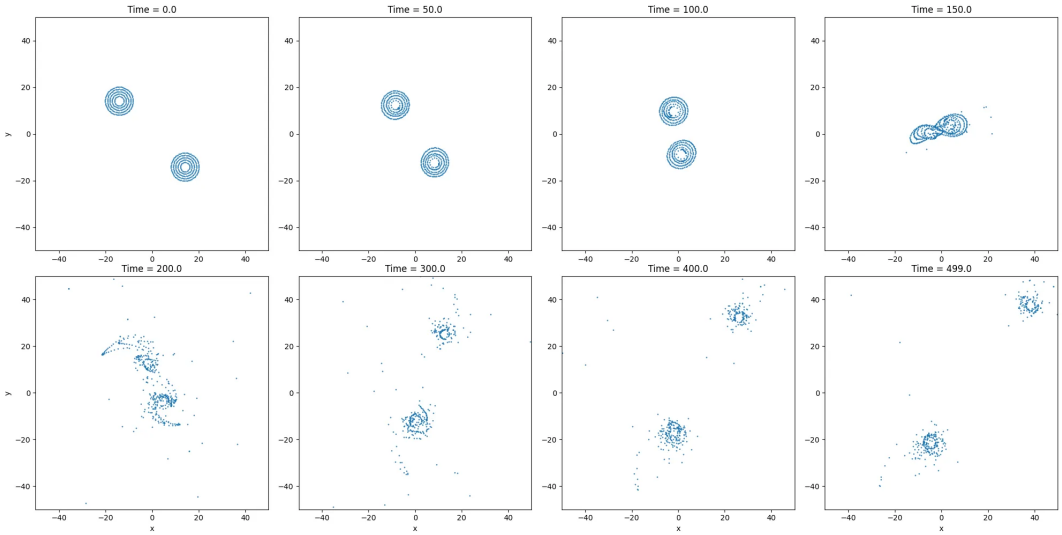

Figure 1: Ejemplos

7 Conclusiones

En conclusión, el trabajo presenta un código en Python dedicado a simular interacciones gravitacionales en sistemas galácticos. Utilizando métodos numéricos, específicamente la integración de ecuaciones diferenciales ordinarias (ODE), el código modela de manera precisa el movimiento de galaxias y estrellas bajo la influencia de la gravedad. Se destacan los parámetros físicos y de animación utilizados en las simulaciones, lo que proporciona una comprensión completa de las condiciones iniciales y los factores que influyen en la evolución de los sistemas galácticos simulados.

La importancia de este enfoque radica en la capacidad de las simulaciones numéricas para representar de manera realista y detallada fenómenos astrofísicos, especialmente en situaciones donde las fuerzas de marea desempeñan un papel crucial. El propósito principal de las simulaciones es extrapolar el conocimiento sobre las fuerzas de marea en el contexto de la colisión de galaxias, buscando una comprensión más profunda de cómo las interacciones gravitacionales afectan la dinámica de sistemas galácticos.

El código se presenta no solo como una herramienta de simulación, sino también como un medio para avanzar en la comprensión de las fuerzas de marea en la colisión de galaxias. La estructura modular del código, junto con parámetros configurables, proporciona flexibilidad para explorar diversos escenarios astrofísicos y condiciones iniciales, contribuyendo a la robustez y versatilidad del código.

Además, se describen diferentes tipos de simulaciones, cada una diseñada para modelar escenarios específicos de interacciones gravitacionales en sistemas galácticos. La teoría matemática detrás del código se basa en principios sólidos, incluyendo ecuaciones diferenciales ordinarias, la Ley de Gravitación Universal de Newton y análisis de colisiones.

En resumen, el trabajo presenta una herramienta valiosa para investigadores interesados en explorar y comprender las complejidades de las interacciones gravitacionales en sistemas galácticos, utilizando enfoques numéricos y matemáticos sólidos.

8 Posibles Extensiones

El código puede ser extendido para incluir interacciones más complejas, como la presencia de materia oscura o la influencia de campos gravitacionales externos. Además, se podrían implementar técnicas de optimización para mejorar la eficiencia computacional en simulaciones de sistemas más grandes y complejos.

DECLARACIONES Y NOTAS COMPLEMENTARIAS

Conflictos de interés:
El autor declara no tener conflictos de interés.

Financiación:
El autor declara que el estudio no recibió financiación.

Agradecimientos:
Se agradece a la comunidad de código abierto por las bibliotecas utilizadas en este proyecto, lo que ha facilitado el desarrollo de esta herramienta de simulación. El autor agradece la contribución de Álvaro Jara y Marcelo Astorga, quienes ayudaron en la maquetación del pre-print.

References

"Matplotlib: Visualization with Python." n.d. Matplotlib. Accessed December 13, 2025. https://matplotlib.org/.

"NumPy: The fundamental package for scientific computing with Python." n.d. NumPy. Accessed December 13, 2025. https://numpy.org/.

"SciPy: Scientific Library for Python." n.d. SciPy. Accessed December 13, 2025. https://www.scipy.org/.